

Research Notes on Q-learning

Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

>> Section 1

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha (r_t + \gamma \max_{a'} Q_t(s_{t+1}, a') - Q_t(s, a))$$
$$Q_{t+1}^A(s_t, a_t) = Q_t^A(s_t, a_t) + \alpha (r_t + \gamma Q_t^B(s_{t+1}, a) - Q_t^A(s_t, a_t))$$
$$Q_{t+1}^A(s_t, a_t) = Q_t^A(s_t, a_t) + \alpha (r_t + \gamma Q_t^B(s_{t+1}, a) - Q_t^A(s_t, a_t))$$
$$Q_{t+1}^B(s_t, a_t) = Q_t^B(s_t, a_t) + \alpha (r_t + \gamma Q_t^A(s_{t+1}, a) - Q_t^B(s_t, a_t))$$

and

>> Section 2

After Δt steps into the future the agent will decide some next step. The weight for this step is calculated as $\gamma^{\Delta t}$, where γ (the discount factor) is a number between 0 and 1 ($0 \leq \gamma \leq 1$). Assuming $\gamma < 1$, it has the effect of valuing rewards received earlier higher than those received later (reflecting the value of a "good start"). γ may also be interpreted as the probability to succeed (or survive) at every step Δt . The algorithm, therefore, has a function that calculates the quality of a state-action combination:

>> Section 3

where R_{t+1} is the reward received when moving from the state S_t to the state S_{t+1} , and α is the learning rate ($0 < \alpha \leq 1$). Note that $Q_{new}(S_t, A_t)$ is the sum of three terms:

>> Section 4

$Q: S \times A \rightarrow R$. Before learning begins, Q is initialized to a possibly arbitrary fixed value (chosen by the programmer). Then, at each time t the agent selects an action A_t , observes a reward R_{t+1} , enters a new state S_{t+1} (that may depend on both the previous state S_t and the selected action), and Q is updated. The core of the algorithm is a Bellman equation as a simple value iteration update, using the weighted average of the current value

and the new information:

>> Section 5

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha (r_t + \gamma \max_{a'} Q_t(s_{t+1}, a') - Q_t(s, a))$$
$$Q_{t+1}^A(s_t, a_t) = Q_t^A(s_t, a_t) + \alpha (r_t + \gamma Q_t^B(s_{t+1}, a) - Q_t^A(s_t, a_t))$$
$$Q_{t+1}^B(s_t, a_t) = Q_t^B(s_t, a_t) + \alpha (r_t + \gamma Q_t^A(s_{t+1}, a) - Q_t^B(s_t, a_t))$$

>> Section 6

0 seconds wait time + 15 seconds fight time On the next day, by random chance (exploration), you decide to wait and let other people depart first. This initially results in a longer wait time. However, less time is spent fighting the departing passengers. Overall, this path has a higher reward than that of the previous day, since the total boarding time is now:

>> Section 7

Reinforcement learning involves an agent, a set of states S , and a set A of actions per state. By performing an action $a \in A$, the agent transitions from state to state. Executing an action in a specific state provides the agent with a reward (a numerical score). The goal of the agent is to maximize its total reward. It does this by adding the maximum reward attainable from future states to the reward for achieving its current state, effectively influencing the current action by the potential future reward. This potential reward is a weighted sum of expected values of the rewards of all future steps starting from the current state. As an example, consider the process of boarding a train, in which the reward is measured by the negative of the total time spent boarding (alternatively, the cost of boarding the train is equal to the boarding time). One strategy is to enter the train door as soon as they open, minimizing the initial wait time for yourself. If the train is crowded, however, then you will have a slow entry after the initial action of entering the door as people are fighting you to depart the train as you attempt to board. The total boarding time, or cost, is then:

>> Section 8

Implementation Q-learning at its simplest stores data in tables. This approach falters with increasing numbers of states/actions since the likelihood of the agent visiting a particular state and performing a particular action is increasingly small.

>> Section 9

Research Notes on Q-learning

Q-learning

Initial conditions (Q0) Since Q-learning is an iterative algorithm, it implicitly assumes an initial condition before the first update occurs. High initial values, also known as "optimistic initial conditions", can encourage exploration: no matter what action is selected, the update rule will cause it to have lower values than the other alternative, thus increasing their choice probability. The first reward r can be used to reset the initial conditions. According to this idea, the first time an action is taken the reward is used to set the value of Q . This allows immediate learning in case of fixed deterministic rewards. A model that incorporates reset of initial conditions (RIC) is expected to predict participants' behavior better than a model that assumes any arbitrary initial condition (AIC). RIC seems to be consistent with human behaviour in repeated binary choice experiments.

>> Section 10

Discount factor The discount factor γ determines the importance of future rewards. A factor of 0 will make the agent "myopic" (or short-sighted) by only considering current rewards, i.e. r_t (in the update rule above), while a factor approaching 1 will make it strive for a long-term high reward. If the discount factor meets or exceeds 1, the action values may diverge. For $\gamma = 1$, without a terminal state, or if the agent never reaches one, all environment histories become infinitely long, and utilities with additive, undiscounted rewards generally become infinite. Even with a discount factor only slightly lower than 1, Q-function learning leads to propagation of errors and instabilities when the value function is approximated with an artificial neural network. In that case, starting with a lower discount factor and increasing it towards its final value accelerates learning.